

Olist E-commerce

Data Warehouse & Business Intelligence

Documentation technique du projet Data Engineering

Auteur

Youssef Khaloufi

Domaine

Data Engineering

Date

Mai 2026

Technologies

Python · PostgreSQL · Docker

Table des matieres

Introduction

Ce document presente la documentation technique complete du projet Data Engineering realise sur le dataset public Olist Brazilian E-Commerce. Il couvre les quatre phases du projet : analyse de la problematique, modelisation dimensionnelle, construction du pipeline ELT et preparation de la couche dashboard.

L'objectif est de construire une base analytique fiable permettant de repondre a des questions metier concretes : suivi des ventes, analyse des paiements, evaluation de la satisfaction client, mesure de la performance logistique et suivi du tunnel d'acquisition marketing des vendeurs. Le projet couvre ainsi la chaine complete : fichiers CSV bruts, nettoyage, modelisation dimensionnelle, chargement du Data Warehouse, datamarts analytiques et controles qualite.

Architecture globale

Le pipeline est organise en quatre couches PostgreSQL successives, chacune avec un perimetre de responsabilite strict :

Couche	Schema PostgreSQL	Role
1	raw	Copie brute des fichiers CSV sans aucune transformation. Point de reprise du pipeline.
2	staging	Nettoyage, typage, standardisation des valeurs texte, deduplication de la geolocalisation, cast booleens.
3	dwh	Modele dimensionnel en flocon. Chargement des dimensions puis des faits. Calcul des metriques derivees.
4	marts	Vues analytiques agreges par domaine metier, directement consommables par le dashboard Streamlit.

Phase 1 — Problematique et analyse des sources

Contexte

Olist est une marketplace bresilienne qui connecte des vendeurs independants a plusieurs canaux de vente en ligne. Le dataset couvre l'ensemble des processus operationnels de la plateforme sur la periode 2016-2018 : commandes, paiements, avis clients, logistique, catalogue produits, vendeurs, geolocalisation et acquisition marketing.

Le point de depart du projet est une problematique metier claire : les donnees Olist existent sous forme de fichiers CSV separes, mais elles ne permettent pas directement d'obtenir une vision consolidee de l'activite. Il faut donc les transformer en un systeme analytique coherent capable de relier commandes, clients, produits, vendeurs, paiements, avis, localisation et acquisition marketing.

Problematique metier

Comment transformer 11 fichiers CSV operationnels et marketing en un Data Warehouse fiable permettant de piloter la performance globale d'Olist, depuis l'acquisition des vendeurs jusqu'aux ventes, paiements, livraisons et avis clients ?

Questions auxquelles le projet doit repondre

Domaine	Question metier	Tables / vues mobilisees
Ventes	Quel est le chiffre d'affaires par mois, statut de commande et categorie produit ?	fact_order_items, fact_orders, dim_date, dim_product_category, marts.sales_overview
Paiements	Quels moyens de paiement representent le plus de valeur et comment les paiements sont-ils repartis ?	fact_payments, dim_payment_type, marts.payment_analysis
Logistique	Quels sont les delais moyens de livraison et quelles commandes sont livrees en retard ?	fact_orders, dim_order_status, marts.delivery_performance
Satisfaction client	Comment les notes clients sont-elles reparties et quels segments concentrent les avis negatifs ?	fact_reviews, fact_orders, dim_product_category, marts.customer_satisfaction
Marketing	Quels canaux generent le plus de leads, de deals conclus et de vendeurs actifs ?	fact_marketing_leads, fact_closed_deals, dim_marketing_origin, marts.marketing_funnel
Geographie	Comment analyser clients et vendeurs par ville, Etat et prefixe postal ?	dim_customer, dim_seller, dim_zip_prefix, dim_city, dim_state
Data Engineering	Comment garantir une donnee tracable, rejouable et controlee de la source jusqu'aux datamarts ?	raw, staging, dwh, marts, quality_checks

Objectifs fonctionnels du Data Warehouse

Objectif	Realisation dans le projet
Centraliser	Regrouper les 11 fichiers Olist dans PostgreSQL avec une couche raw complete.
Nettoyer	Standardiser les dates, textes, booleens, codes postaux et dedupliquer la geolocalisation.
Modeliser	Construire un schema en flocon avec dimensions partagees et plusieurs tables de faits.
Mesurer	Calculer les indicateurs utiles : revenu, fret, delais, retard, score avis, conversion marketing.
Exposer	Creer des vues marts directement exploitables par un futur dashboard Streamlit.

Inventaire des sources

Les 11 fichiers couvrent trois domaines distincts : les operations e-commerce (commandes, paiements, avis), le catalogue (produits, categories, vendeurs, geolocalisation) et le CRM commercial (leads marketing, deals conclus).

Fichier CSV	Lignes	Description
-------------	--------	-------------

olist_orders_dataset.csv	99 441	Commandes avec timestamps complets du cycle de vie (achat, approbation, expedition, livraison, estimation).
olist_order_items_dataset.csv	112 650	Articles commandes. Cle composite (order_id, order_item_id). Contient prix unitaire et frais de port par item.
olist_order_payments_dataset.csv	103 886	Paielements. Une commande peut avoir plusieurs lignes de paiement (multi-modes, paiement fractionne).
olist_order_reviews_dataset.csv	104 719	Avis clients avec score 1-5, titre, message, date de creation et de reponse.
olist_customers_dataset.csv	99 441	Clients avec double identifiant : customer_id lie a la commande, customer_unique_id stable cross-commandes.
olist_products_dataset.csv	32 951	Catalogue produits avec categorie (en portugais), dimensions physiques et nombre de photos.
olist_sellers_dataset.csv	3 095	Vendeurs de la marketplace avec localisation (code postal, ville, etat).
olist_geolocation_dataset.csv	~1 000 000	Table de reference geographique par prefixe postal. Un meme prefixe peut avoir plusieurs lignes avec des coordonnees legerement differentes.
product_category_name_translation.csv	70	Table de correspondance entre les noms de categories en portugais et leur traduction en anglais.
olist_marketing_qualified_leads_dataset.csv	8 000	Leads marketing qualifies (MQL) avec canal d'origine et date de premier contact.
olist_closed_deals_dataset.csv	842	Deals conclus : leads convertis en vendeurs actifs. Contient des informations metier sur le profil du vendeur.

Enjeux techniques identifiés

L'analyse des sources revele plusieurs contraintes techniques a traiter dans la couche staging :

Probleme	Solution appliquee dans staging
Geolocalisation multiple	Un meme code postal peut avoir des centaines de lignes avec des coordonnees differentes. Solution : agregation par moyenne des lat/lng, puis selection de la ville la plus frequente par ROW_NUMBER() OVER PARTITION BY zip_prefix.
Codes postaux mal formats	LPAD(TRIM(zip_prefix), 5, '0') pour standardiser a 5 caracteres avec zero-padding.
Valeurs booléennes texte	Les colonnes has_company et has_gtin arrivent en TEXT avec des valeurs 'true', '1', 'sim', 'yes'. Cast conditionnel avec CASE WHEN LOWER(TRIM(...)) IN (...) THEN TRUE.
Noms de villes inconsistants	LOWER(TRIM()) applique systematiquement pour unifier la casse avant les jointures dans la dimension dim_city.
Doublons sur les avis	La table olist_order_reviews peut contenir plusieurs review_id pour un meme order_id. La cle primaire est sur review_id dans la table de faits.
NULL sur les timestamps	Les timestamps order_approved_at, order_delivered_* peuvent etre NULL (commandes en attente ou annulees). Geres via COALESCE et indicateurs booléens is_delivered dans fact_orders.

Phase 2 — Modelisation dimensionnelle

Choix du schema en flocon

La diversite des processus metier identifies en phase 1 impose plusieurs tables de faits independantes. Un schema en etoile simple aurait entraine une duplication des attributs geographiques (ville, etat, code postal) entre les dimensions client et vendeur.

Le schema en flocon (snowflake schema) a ete retenu pour les raisons suivantes :

Critere	Justification
Dimensions partagees	dim_zip_prefix, dim_city, dim_state sont referencees a la fois par dim_customer et dim_seller. Le flocon evite la redundance.
Normalisation geographique	La hierarchie state -> city -> zip_prefix est naturelle et permet des analyses a differents niveaux de granularite.
Coherence des references	Une modification du nom d'une ville est propagee automatiquement a tous les clients et vendeurs de cette ville.
Performance acceptable	A l'echelle du dataset Olist (~100k commandes), les jointures supplementaires du flocon n'ont pas d'impact significatif. Les index sur les foreign keys couvrent les cas d'usage du dashboard.

Tables de faits

Table	Grain	Mesures et indicateurs
fact_orders	1 commande	approval_delay_hours, days_to_carrier, days_to_customer, estimated_delivery_days, delivery_delay_days, is_delivered_on_time (BOOLEAN), is_delivered (BOOLEAN), order_count
fact_order_items	1 article commande	item_price, freight_value, total_item_value (= price + freight), item_count
fact_payments	1 paiement	payment_installments, payment_value, payment_count
fact_reviews	1 avis client	review_score, review_response_time_hours, has_review_title, has_review_message, is_positive_review (score >= 4), is_neutral_review (score = 3), is_negative_review (score <= 2)
fact_marketing_leads	1 lead MQL	lead_count (= 1, additive)
fact_closed_deals	1 deal conclu	declared_product_catalog_size, declared_monthly_revenue, is_active_seller, deal_count

Dimensions

Dimension	Attributs cles
dim_date	date_key (YYYYMMDD), full_date, day_of_month, day_of_week, day_name, week_of_year, month_number, month_name, quarter_number, year_number, is_weekend
dim_customer	customer_id, customer_unique_id, zip_prefix_key (FK), city_key (FK), state_key (FK)
dim_seller	seller_id, zip_prefix_key (FK), city_key (FK), state_key (FK)
dim_product	product_id, product_category_key (FK), product_name_length, product_description_length, product_photos_qty, product_weight_g, dimensions (l/h/w cm)

dim_product_category	product_category_name (PT), product_category_name_english (EN)
dim_zip_prefix	zip_prefix, city_key (FK), state_key (FK), latitude, longitude
dim_city	city_name, state_key (FK) — UNIQUE(city_name, state_key)
dim_state	state_code (2 lettres), state_name (nom complet en portugais)
dim_order_status	order_status — valeurs : delivered, shipped, canceled, processing...
dim_payment_type	payment_type — valeurs : credit_card, boleto, voucher, debit_card
dim_marketing_origin	origin_name — valeurs : organic_search, social, direct_traffic, email...
dim_sales_rep	sales_rep_id, sales_role — UNIQUE(sales_rep_id, sales_role). Deux rôles : SDR et SR.

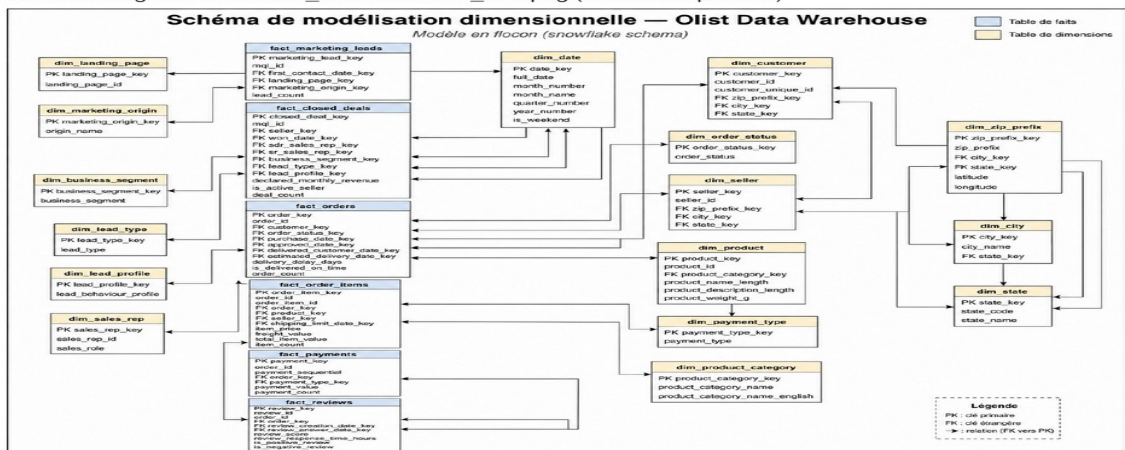
Schema de modelisation dimensionnelle

3. Images

docs/images/modelisation_dimensionnelle_olist.png

Type: image | Source: /mnt/e/projects/olist-data-warehouse-bi/docs/images/modelisation_dimensionnelle_olist.png

FILE: docs/images/modelisation_dimensionnelle_olist.png (embedded preview)



Schema de modelisation dimensionnelle — Olist Data Warehouse (snowflake schema)

Phase 3 — Pipeline ELT

Strategie de chargement

Le pipeline suit une strategie ELT (Extract → Load → Transform) : les donnees brutes sont chargees en premier dans le schema raw sans aucune transformation, puis transformees en SQL directement dans PostgreSQL. Cette approche presente plusieurs avantages : la couche raw permet de rejouer n'importe quelle etape sans re-telechargement, et les transformations SQL sont plus performantes que des transformations Python sur des volumes de cette taille.

Couche RAW — Ingestion

Mecanisme de chargement

Le chargement des CSV vers la couche raw utilise la commande PostgreSQL COPY FROM STDIN via `psycpg2.cursor.copy_expert()`. Cette approche est significativement plus rapide qu'un INSERT ligne par ligne : pour 1 million de lignes de geolocalisation, le COPY prend quelques secondes contre plusieurs minutes en mode INSERT.

```
COPY raw.olist_geolocation (geolocation_zip_code_prefix, geolocation_lat, geolocation_lng,
                           geolocation_city, geolocation_state)
FROM STDIN
WITH (FORMAT CSV, HEADER TRUE, NULL '', ENCODING 'UTF8')
```

Avant chaque chargement, toutes les tables raw sont tronquees avec `RESTART IDENTITY CASCADE` pour garantir l'idempotence du pipeline. Un rechargement complet est toujours possible sans effets de bord.

Couche STAGING — Transformations

Standardisation geographique

La geolocalisation est le traitement le plus complexe de la couche staging. La table brute contient ~1 million de lignes pour seulement ~19 000 prefixes postaux uniques, avec des coordonnees et des noms de villes variables pour un meme prefixe. La transformation applique deux etapes :

```
-- Etape 1 : moyenne des coordonnees par prefixe
geo_avg AS (
  SELECT zip_prefix, AVG(latitude) AS latitude, AVG(longitude) AS longitude
  FROM cleaned GROUP BY zip_prefix
),
-- Etape 2 : ville la plus frequente pour ce prefixe
city_rank AS (
  SELECT zip_prefix, city_name, state_code,
  ROW_NUMBER() OVER (PARTITION BY zip_prefix
    ORDER BY COUNT(*) DESC, city_name, state_code) AS rn
  FROM cleaned GROUP BY zip_prefix, city_name, state_code
)
```

Cast des booleens

Les colonnes `has_company` et `has_gtin` arrivent en TEXT dans le fichier brut avec des valeurs heterogenes. La transformation staging les convertit en BOOLEAN via un CASE conditionnel tolerant :


```

CASE WHEN LOWER(TRIM(has_company)) IN ('true','1','yes','y','sim') THEN TRUE
      WHEN LOWER(TRIM(has_company)) IN ('false','0','no','n','nao') THEN FALSE
      ELSE NULL
END AS has_company

```

Couche DWH — Chargement dimensionnel

Ordre de chargement

Les dimensions doivent impérativement être chargées avant les tables de faits, et les dimensions de référence géographique avant les autres dimensions. L'ordre appliqué est le suivant :

Ordre	Table	Dependance
1	dim_date	Générée par GENERATE_SERIES couvrant toutes les dates présentes dans staging. Aucune dépendance.
2	dim_state	Source : UNION des state_code de stg_geolocation, stg_customers, stg_sellers. CASE WHEN pour les noms complets.
3	dim_city	Dependance : dim_state (pour state_key). Cle unique sur (city_name, state_key).
4	dim_zip_prefix	Dependance : dim_city, dim_state. Coordonnées GPS issues de stg_geolocation.
5	dim_customer / dim_seller	Dependance : dim_zip_prefix, dim_city, dim_state.
6	dim_product_category / dim_product	dim_product dépend de dim_product_category pour product_category_key.
7	Petites dimensions	dim_order_status, dim_payment_type, dim_marketing_origin, dim_landing_page, etc. SELECT DISTINCT avec COALESCE('unknown').
8	fact_marketing_leads	Première table de faits. Dependance : dim_date, dim_landing_page, dim_marketing_origin.
9	fact_closed_deals	Dependance : dim_seller, dim_date, dim_sales_rep, dim_business_segment, dim_lead_type, dim_lead_profile.
10	fact_orders	Table de faits centrale. Calcul des métriques temporelles via EXTRACT(EPOCH FROM ...).
11	fact_order_items	Dependance : fact_orders (order_key), dim_product, dim_seller.
12	fact_payments	Dependance : fact_orders (order_key), dim_payment_type.
13	fact_reviews	Dependance : fact_orders (order_key). Dernier à charger.

Calcul des métriques de livraison

Les indicateurs de performance logistique sont calculés directement dans l'INSERT de fact_orders par des opérations sur les timestamps :

```

ROUND((EXTRACT(EPOCH FROM (order_delivered_customer_ts - order_purchase_ts))
      / 86400.0)::numeric, 2) AS days_to_customer,

ROUND((EXTRACT(EPOCH FROM (order_delivered_customer_ts - order_estimated_delivery_ts))
      / 86400.0)::numeric, 2) AS delivery_delay_days,

CASE WHEN order_delivered_customer_ts IS NULL
      OR order_estimated_delivery_ts IS NULL THEN NULL
      ELSE order_delivered_customer_ts <= order_estimated_delivery_ts

```

```
END AS is_delivered_on_time
```

Couche MARTS — Vues analytiques

Les marts sont des vues PostgreSQL (CREATE OR REPLACE VIEW) agregant les tables du schema dwh. Elles constituent la seule interface exposee au dashboard — celui-ci ne requete jamais directement les tables de faits ou dimensions.

Vue	Tables jointures	Indicateurs exposes
marts.sales_overview	fact_orders, fact_order_items, dim_date, dim_order_status	order_count, item_count, gross_item_value, gross_freight_value, gross_revenue
marts.sales_by_category	fact_order_items, fact_orders, dim_date, dim_product, dim_product_category	item_count, order_count, item_revenue, freight_revenue, total_revenue par categorie
marts.delivery_performance	fact_orders, dim_date, dim_order_status	delivered_orders, avg_days_purchase_to_delivery, avg_delivery_delay, late_delivery_rate_pct
marts.customer_satisfaction	fact_reviews, fact_orders, fact_order_items, dim_product, dim_product_category, dim_date	review_count, avg_review_score, negative_review_rate_pct par categorie
marts.payment_analysis	fact_payments, fact_orders, dim_date, dim_payment_type	payment_count, total_payment_value, avg_payment_value par type et nombre de versements
marts.marketing_funnel	fact_marketing_leads, fact_closed_deals, dim_date, dim_marketing_origin	mql_count, won_deal_count, active_seller_deal_count, lead_to_deal_conversion_rate_pct

Orchestration — DAG Airflow

Un DAG Apache Airflow optionnel est fourni pour les environnements de production. Il encode les dependances entre etapes et permet une reexecution partielle en cas d'echec.

```
create_schema >> load_raw_csvs >> build_staging >> load_dwh >> build_marts >>
run_quality_checks
```

Chaque tache est un BashOperator appelant `python -m src.run_etl --step <etape>`. La tache `run_quality_checks` utilise le flag `--fail-fast-quality` pour interrompre le DAG en cas d'echec d'un controle.

Stack technique

Composant	Version	Role dans le projet
Python	3.11+	Orchestration du pipeline, scripts ETL, controles qualite
PostgreSQL	16	Base de donnees relationnelle hebergeant les 4 couches (raw, staging, dwh, marts)
SQLAlchemy	2.x	Couche d'abstraction Python vers PostgreSQL. Gestion des connexions et execution des statements SQL.
psycopg2-binary	2.9+	Acces bas niveau pour <code>copy_expert()</code> (COPY FROM STDIN). Significativement plus rapide que l'ORM pour les bulk loads.
pandas	2.x	Lecture et inspection des fichiers sources. Non utilise dans le pipeline de chargement (remplace par COPY).

python-dotenv	1.x	Chargement de DATABASE_URL et RAW_DATA_DIR depuis le fichier .env.
Docker / Compose	—	Conteneurisation de PostgreSQL 16. Volume nomme postgres_data pour la persistance.
Apache Airflow	2.x	Orchestration optionnelle. DAG fourni dans airflow/dags/olist_etl_dag.py.

Contrôles qualité

Huit contrôles automatisés sont exécutés après le chargement du Data Warehouse via `src/quality_checks.py`. Chaque contrôle est une requête SQL dont le résultat est évalué par un prédicat Python.

Nom du contrôle	Requête SQL	Attendu	Résultat
raw_orders_loaded	SELECT COUNT(*) FROM raw.olist_orders	> 0	PASS
dwh_fact_orders_loaded	SELECT COUNT(*) FROM dwh.fact_orders	> 0	PASS
fact_order_items_missing_order_key	SELECT COUNT(*) FROM dwh.fact_order_items WHERE order_key IS NULL	= 0	PASS
fact_order_items_missing_product_key	SELECT COUNT(*) FROM dwh.fact_order_items WHERE product_key IS NULL	= 0	PASS
fact_order_items_missing_seller_key	SELECT COUNT(*) FROM dwh.fact_order_items WHERE seller_key IS NULL	= 0	PASS
fact_payments_missing_order_key	SELECT COUNT(*) FROM dwh.fact_payments WHERE order_key IS NULL	= 0	PASS
fact_reviews_missing_order_key	SELECT COUNT(*) FROM dwh.fact_reviews WHERE order_key IS NULL	= 0	PASS
reviews_outside_1_5	SELECT COUNT(*) FROM dwh.fact_reviews WHERE review_score NOT BETWEEN 1 AND 5	= 0	PASS

Résultat final : 8 / 8 contrôles réussis.

Resultats du pipeline

Volume de donnees chargees

Table DWH	Lignes
fact_orders	99 441
fact_order_items	112 650
fact_payments	103 886
fact_reviews	99 224
fact_marketing_leads	8 000
fact_closed_deals	842
dim_customer	99 441
dim_seller	3 095
dim_product	32 951
dim_date	1 314

Indicateurs cles

Revenus Valeur totale articles : 13 591 643,70 R\$ Frais de livraison : 2 251 909,54 R\$ Revenu brut total : 15 843 553,24 R\$	Paielements Carte de credit : 12 542 084,19 R\$ Boleto bancario : 2 869 361,27 R\$ Voucher : 379 436,87 R\$ Carte de debit : 217 989,79 R\$
Avis clients Score 5 : 57 328 avis (57,8%) Score 4 : 19 142 avis (19,3%) Score 3 : 8 179 avis (8,2%) Score 2 : 3 151 avis (3,2%) Score 1 : 11 424 avis (11,5%)	Tunnel marketing (extrait 2018) Organic search jan. : 412 leads, 63 deals Organic search avr. : 349 leads, 59 deals Social avr. : 325 leads, 16 deals

Phase 4 — Dashboard Streamlit (en cours)

Les vues du schema marts ont ete concues des la phase de modelisation pour etre directement consommables par un dashboard sans aucune modification du pipeline. La phase 4 consiste a developper une interface Streamlit connectee a ces vues via SQLAlchemy.

Pages prevues

Page	Vue marts utilisee	Contenu
Vue globale des ventes	sales_overview, sales_by_category	CA mensuel par statut, top 10 categories, evolution temporelle
Analyse des paiements	payment_analysis	Repartition par type, distribution des versements, valeur moyenne
Satisfaction client	customer_satisfaction	Score moyen par mois et categorie, taux d'avis negatifs
Performance logistique	delivery_performance	Delai moyen, taux de livraison en retard par mois
Tunnel marketing	marketing_funnel	MQL vs deals conclus, taux de conversion par canal d'acquisition

Principe de connexion

Le dashboard se connecte exclusivement au schema marts. Aucune requete directe sur dwh, staging ou raw. Cette separation garantit que l'evolution du modele interne du DWH ne casse pas le dashboard.

```
# Connexion Streamlit -> marts
engine = create_engine(os.getenv('DATABASE_URL'))
df = pd.read_sql('SELECT * FROM marts.sales_overview', engine)
```

Etat d'avancement

Phase	Etape	Statut
Problematique	Analyse des sources et documentation	Termine
Modelisation	Schema dimensionnel en flocon	Termine
ETL	Creation des schemas PostgreSQL	Termine
ETL	Chargement couche raw	Termine
ETL	Transformation couche staging	Termine
ETL	Chargement Data Warehouse	Termine
ETL	Creation des marts analytiques	Termine
ETL	Controles qualite (8/8 PASS)	Termine
Dashboard	Developpement Streamlit	En cours

Sources et references

Dataset e-commerce : [Olist Brazilian E-Commerce Public Dataset](#)

Dataset marketing : [Olist Marketing Funnel Dataset](#)

Les deux datasets sont publiés sous licence CC BY-NC-SA 4.0.

Ce projet est open-source sous licence MIT.